Tema 2 - CSS y construcción de interfaces web



2.3 - Flujo, visualización y posicionamiento con CSS

Ya sabemos cuánto mide una caja. Ahora necesitamos saber dónde aparece.

En el apartado anterior aprendimos a controlar el tamaño de las cajas CSS mediante dimensiones, unidades, funciones, padding, border, margin y box-sizing. Pero dos cajas con exactamente las mismas dimensiones pueden aparecer en lugares completamente distintos.

Ahora debemos responder a otras preguntas:

- ¿Por qué algunos elementos comienzan en una nueva línea y otros permanecen dentro del texto?
- ¿Qué ocurre con el resto del documento cuando ocultamos un elemento?
- ¿Qué sucede cuando el contenido no cabe dentro de una caja?
- ¿Podemos desplazar una caja sin modificar la posición de las demás?
- ¿Respecto a qué punto se calcula `top: 20px`?
- ¿Por qué un elemento puede permanecer visible mientras hacemos scroll?
- ¿Qué ocurre cuando dos elementos ocupan la misma zona?

Para responder a estas preguntas debemos comprender tres conceptos relacionados:



Idea fundamental: Antes de mover una caja debemos comprender dónde la habría colocado el navegador si no hubiéramos intervenido.

El flujo normal: el punto de partida del navegador

Si no aplicamos un sistema especial de posicionamiento o maquetación, el navegador distribuye el contenido siguiendo las reglas del flujo normal. Consideremos este HTML:

```
<h1>Aula Abierta</h1>
<p>Formación práctica para profesionales digitales.</p>
<p>Consulta nuestros <a href="cursos.html">cursos disponibles</a>.</p>
```

El navegador no coloca estos elementos al azar. Los elementos que generan cajas de bloque participan normalmente uno debajo de otro dentro del flujo. El contenido en línea se distribuye dentro de líneas de texto. Por tanto:

`<h1>`

Genera normalmente una caja de bloque.

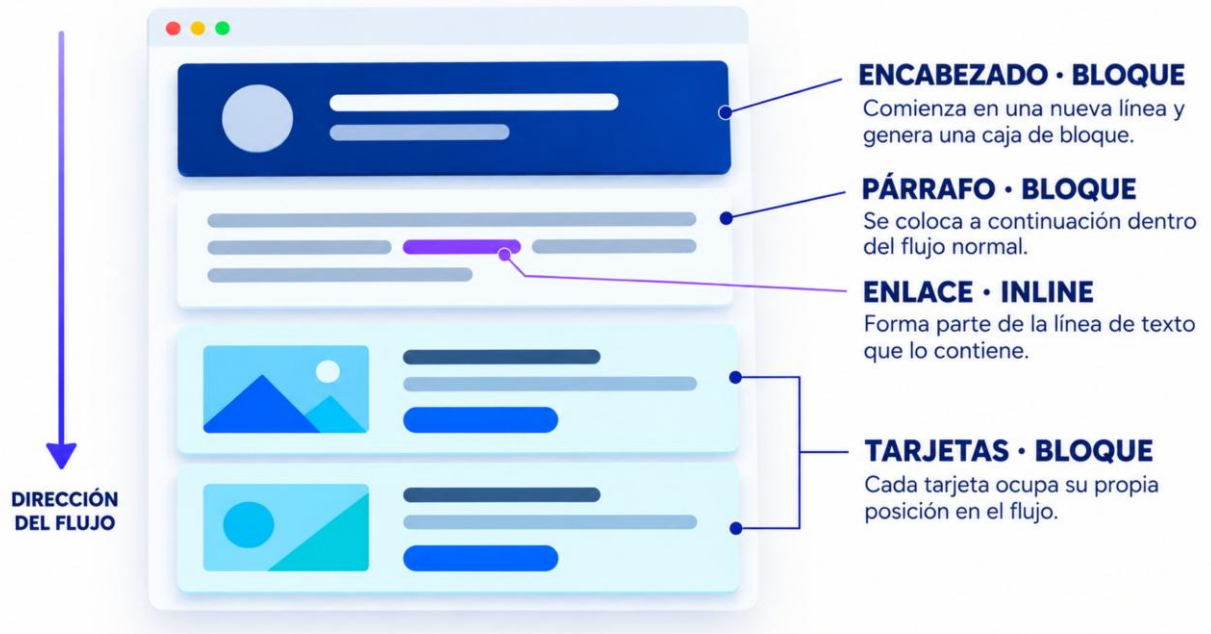
`<p>`

Genera normalmente una caja de bloque.

`<a>`

Participa normalmente dentro de la línea de texto que lo contiene.

El navegador calcula la posición de cada caja teniendo en cuenta: el orden del documento, el tipo de caja, sus dimensiones, padding, border, margin, y las cajas que aparecen antes.



Idea clave: El flujo normal es la posición de partida. Cuando más adelante utilicemos `relative`, `absolute`, `fixed`, `sticky` o `float`, podremos comprender qué ha cambiado respecto a ese comportamiento inicial.

`display`: controlar qué tipo de caja genera un elemento

La propiedad `display` determina cómo genera cajas un elemento y cómo participa en la distribución. En este apartado necesitamos conocer cinco valores fundamentales.

`display: block`

```
.bloque { display: block; }
```

Genera una caja de bloque. Normalmente comienza en una nueva línea. Cuando `width` es `auto`, suele utilizar el espacio horizontal disponible dentro de su contenedor. Podemos controlar sus dimensiones mediante `width` y `height`.

`display: inline`

```
.en-linea { display: inline; }
```

Participa dentro de una línea junto a otros contenidos. En los elementos inline no reemplazados, `width` y `height` no controlan la caja del mismo modo que en un elemento de bloque.

`display: inline-block`

```
.etiqueta { display: inline-block; }
```

La caja participa externamente dentro de una línea, pero permite controlar dimensiones, padding y otros aspectos del modelo de cajas de forma similar a una caja de bloque.

`display: list-item`

```
.elemento { display: list-item; }
```

Genera una caja principal y un marcador de lista. Es el comportamiento utilizado normalmente por elementos `<li>`.

`display: none`

```
.oculto { display: none; }
```

El elemento deja de generar cajas. Su espacio desaparece de la distribución.

Importante: La etiqueta HTML no fija para siempre la forma en que debe mostrarse un elemento. CSS puede modificar su comportamiento de visualización mediante `display`.

`inline-block`: permanecer en una línea conservando una caja controlable

`display: inline-block` resulta útil cuando queremos que varias cajas participen en una misma línea, pero necesitamos aplicarles padding, dimensiones o bordes de forma controlada. Consideremos tres etiquetas:

```
<!-- HTML -->
<span class="etiqueta">HTML</span>
<span class="etiqueta">CSS</span>
<span class="etiqueta">Accesibilidad</span>
```

```
/* CSS */
.etiqueta {
  display: inline-block;
  padding: 0.4em 0.8em;
  border: 1px solid currentColor;
  border-radius: 0.5rem;
}
```

Las tres cajas permanecen dentro de la misma línea siempre que exista espacio suficiente.

vertical-align

Los elementos `inline` e `inline-block` participan en una línea y pueden necesitar ajustar su alineación vertical respecto a los demás elementos de esa línea.

```
.icono {
  display: inline-block;
  vertical-align: middle;
}
```

`baseline`

Alineación respecto a la línea base.

`middle`

Alinea el elemento aproximadamente con la zona media de la línea. No constituye un centrado vertical general.

`top`

Alinea la caja con la parte superior de la línea.

`bottom`

Alinea la caja con la parte inferior de la línea.

⚠ **Importante:** `vertical-align` no es un sistema general para centrar verticalmente una caja dentro de una página o un contenedor. Actúa dentro del contexto de elementos en línea y también tiene aplicación en celdas de tabla. Más adelante veremos técnicas específicas para centrar cajas.

`display: none`, `visibility: hidden` y `opacity: 0` producen resultados distintos

Podemos hacer que un elemento deje de verse de varias formas, pero no todas modifican de la misma manera la distribución.

CASO 1 — display: none

```
.elemento { display: none; }
```

El elemento no genera ninguna caja. Su espacio desaparece. Los elementos posteriores pueden ocupar la posición que ha quedado libre.

CASO 2 — visibility: hidden

```
.elemento { visibility: hidden; }
```

La caja continúa formando parte de la distribución. No se muestra visualmente. El espacio que ocupaba permanece reservado.

CASO 3 — opacity: 0

```
.elemento { opacity: 0; }
```

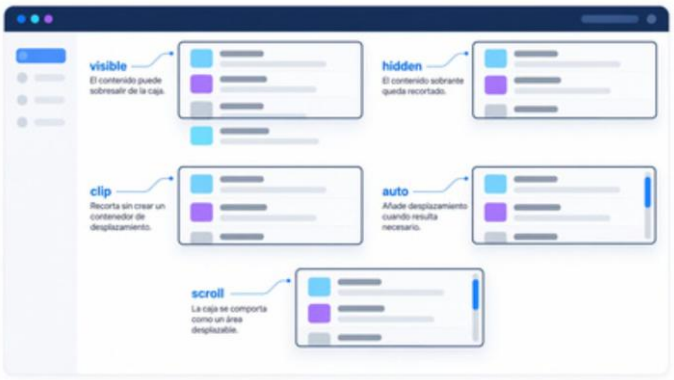
La caja continúa existiendo y ocupando espacio. El elemento se representa con opacidad cero. Puede seguir participando en la interacción y en el foco dependiendo del tipo de elemento y de otras propiedades. Por eso `opacity: 0` no debe utilizarse por sí sola para considerar que un control ha dejado de existir o está deshabilitado.



`overflow`: decidir qué ocurre cuando el contenido supera los límites de su caja

En el apartado anterior vimos que una caja puede tener dimensiones máximas. Por ejemplo:

```
.panel {
  width: 20rem;
  height: 8rem;
}
```



Pero el contenido real puede necesitar más espacio. Cuando el contenido supera el espacio disponible aparece un desbordamiento u *overflow*. CSS permite decidir cómo tratarlo.

`overflow: visible`

```
.panel { overflow: visible; }
```

Comportamiento inicial habitual. El contenido puede pintarse fuera de los límites de la caja.

`overflow: hidden`

```
.panel { overflow: hidden; }
```

El contenido que supera los límites queda recortado. La caja puede seguir siendo un contenedor de desplazamiento aunque no muestre barras de scroll.

`overflow: clip`

```
.panel { overflow: clip; }
```

Recorta el contenido. A diferencia de `hidden`, no crea un contenedor de desplazamiento.

`overflow: auto`

```
.panel { overflow: auto; }
```

Permite desplazamiento cuando el contenido lo necesita.

`overflow: scroll`

```
.panel { overflow: scroll; }
```

Establece un contenedor desplazable. Dependiendo de la plataforma, las barras de desplazamiento pueden mostrarse permanentemente o utilizar el comportamiento que determine el navegador.

⚠ **Idea fundamental:** No debemos aplicar `overflow: hidden` simplemente para esconder un problema de maquetación. Primero debemos comprender por qué se está produciendo el desbordamiento.

Controlar el `overflow` por ejes y el comportamiento del desplazamiento

Podemos controlar cada eje de forma independiente.

`overflow-x`

Controla el desbordamiento horizontal.

`overflow-y`

Controla el desbordamiento vertical.

```
.panel {
  overflow-x: hidden;
  overflow-y: auto;
}
```

En este caso: el contenido horizontal que desborda queda recortado; el contenido vertical puede desplazarse cuando sea necesario.

`scroll-behavior: smooth`

```
html {
  scroll-behavior: smooth;
}
```

Esta propiedad permite que determinados desplazamientos iniciados mediante navegación o mecanismos de scroll se realicen de forma suave en lugar de producir un salto inmediato. No modifica directamente la velocidad con la que el usuario mueve la rueda del ratón o realiza un gesto táctil.



Personalización básica de la barra

CSS dispone de propiedades estandarizadas para aplicar una personalización limitada.

```
.panel {
  scrollbar-width: thin;
  scrollbar-color: #64748B #E2E8F0;
}
```

`scrollbar-width`

Permite indicar: `auto`, `thin`, `none`. No conviene utilizar `none` sin una razón justificada porque eliminamos una pista visual importante de que existe más contenido.

`scrollbar-color`

Recibe dos colores: el primer color corresponde al control deslizante y el segundo a la pista. La personalización debe ser moderada. Una barra de desplazamiento es un control de interfaz del navegador y debe seguir siendo fácilmente reconocible y utilizable.

`position`: cambiar la relación entre una caja y su posición

Hasta ahora hemos estudiado cómo genera cajas un elemento y cómo se comporta su contenido. La propiedad `position` permite controlar de otra manera la posición de una caja. Estudiaremos cinco valores.



- `static`

  La caja utiliza el posicionamiento normal correspondiente a su flujo.

- `relative`

  La caja conserva su lugar en el flujo, pero puede desplazarse visualmente respecto a su posición original.

- `absolute`

  La caja sale del flujo normal y se posiciona utilizando un bloque contenedor como referencia.

- `fixed`

  La caja sale del flujo y normalmente permanece situada respecto al viewport.

- `sticky`

  La caja participa inicialmente en el flujo, pero puede mantenerse a una determinada distancia del borde de su zona de desplazamiento mientras hacemos scroll.

Antes de utilizar `position` debemos preguntarnos:

¿La caja debe conservar su espacio original?, ¿Debe afectar a las demás cajas?, ¿Respecto a qué referencia debe posicionarse?, ¿Debe desplazarse junto con el documento?, ¿Debe permanecer visible durante el scroll?

`position: static` y las propiedades de desplazamiento

El valor inicial de `position` es `static`.

```
.elemento {
  position: static;
}
```

La caja se sitúa de acuerdo con las reglas normales de distribución. Con `static`, las propiedades de desplazamiento no producen el efecto de posicionamiento que buscamos. Por ejemplo:

```
.elemento {
  position: static;
  top: 2rem;
  left: 2rem;
}
```

`top` y `left` no desplazan esta caja. Estas propiedades actúan sobre cajas posicionadas mediante los esquemas correspondientes.

Las cuatro propiedades físicas de desplazamiento

`top`	`right`
`bottom`	`left`

Expresan un desplazamiento o una separación respecto al lado correspondiente de la referencia utilizada para posicionar la caja.

El shorthand `inset`

```
.elemento {
  position: absolute;
  inset: 1rem;
}
```

Equivale en este caso a:

```
.elemento {
  position: absolute;
  top: 1rem;
  right: 1rem;
  bottom: 1rem;
  left: 1rem;
}
```

También podemos escribir `inset: 1rem 2rem;` que aplica: 1rem arriba y abajo; 2rem a izquierda y derecha.

Idea clave: `top`, `right`, `bottom` y `left` no son simples órdenes de «mover». Su significado depende del tipo de posicionamiento y del bloque utilizado como referencia.

`relative`: mover visualmente una caja sin liberar su espacio

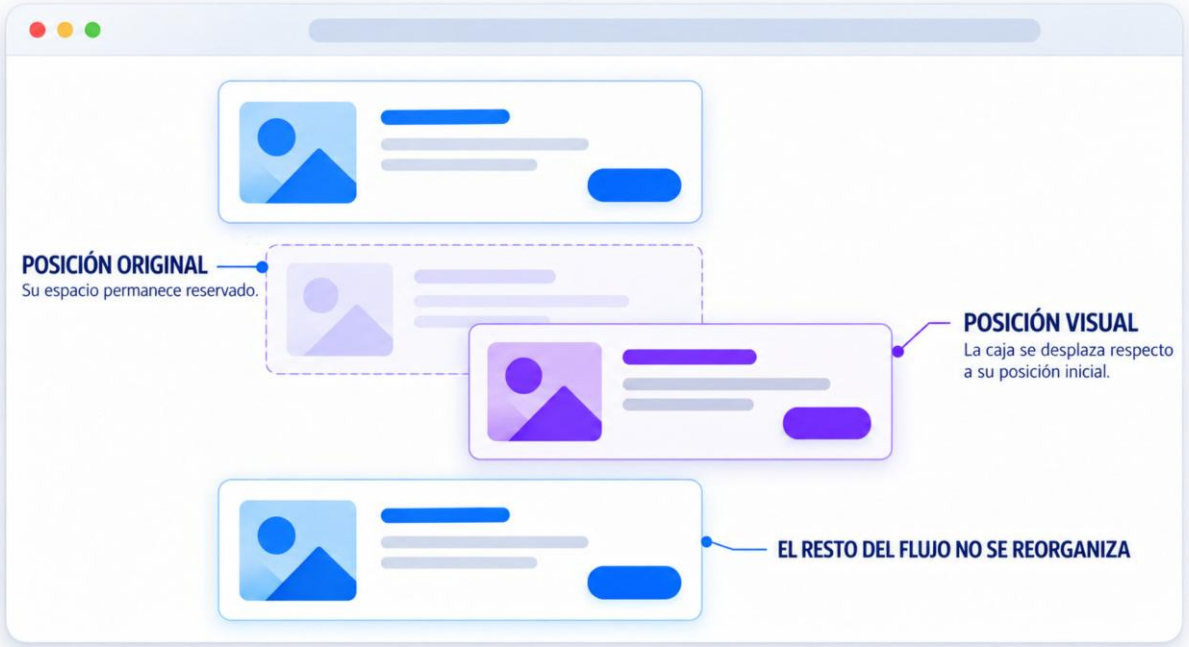
Con `position: relative`, el navegador realiza primero la distribución normal del elemento. Después puede desplazar visualmente la caja.

```
.aviso {
  position: relative;
  top: 1rem;
  left: 2rem;
}
```

Debemos distinguir dos posiciones:

Posición original	Posición visual
Es el lugar calculado por el flujo normal.	Es el lugar en el que finalmente se muestra la caja después de aplicar los offsets.

La posición original continúa reservada. Por eso las demás cajas no reorganizan automáticamente su distribución para ocupar el hueco visual.



Idea fundamental: `relative` modifica la posición visual de la caja, pero no elimina su posición original del flujo.

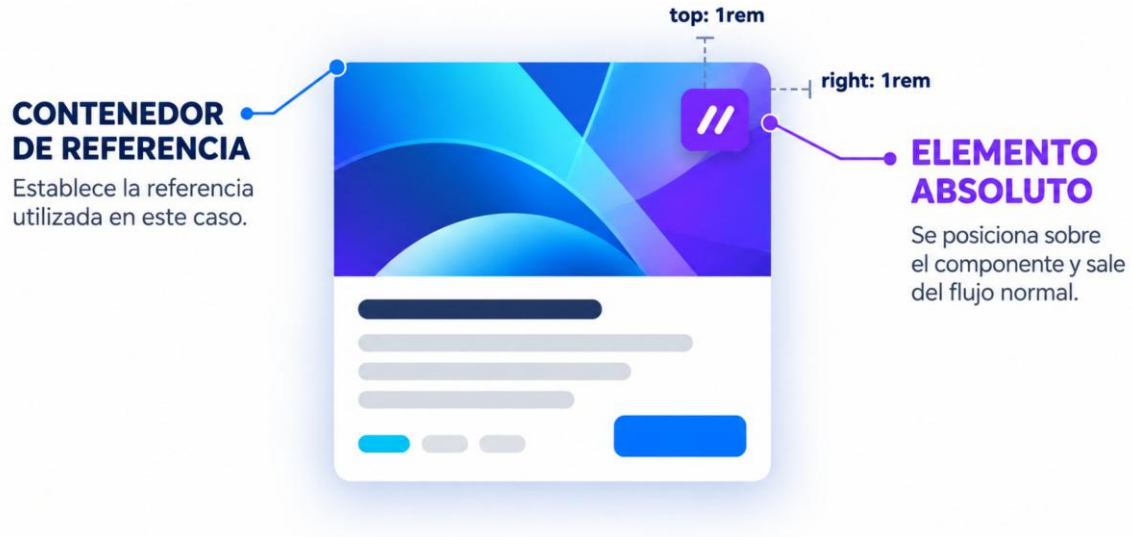
`relative` también puede establecer una referencia para un descendiente

`position: relative` no se utiliza únicamente para desplazar elementos. Uno de sus usos más frecuentes consiste en establecer una referencia de posicionamiento para un descendiente con `position: absolute`.

```
.tarjeta {
  position: relative;
}

.etiqueta {
  position: absolute;
  top: 1rem;
  right: 1rem;
}
```

En este ejemplo no hemos aplicado ningún desplazamiento a `.tarjeta`. Su posición visual no cambia. Sin embargo, `position: relative` hace que podamos utilizar la tarjeta como referencia de posicionamiento para el elemento absoluto situado en su interior.



La combinación habitual:

CONTENEDOR → `position: relative;`
DESCENDIENTE → `position: absolute;`

⚠ **Importante:** No debemos memorizar «absolute se posiciona respecto a su padre». Eso no es siempre cierto. Una caja `absolute` utiliza su bloque contenedor de posicionamiento. Establecer `position: relative` en el contenedor que queremos utilizar como referencia es una forma habitual y clara de conseguirlo. Otros mecanismos CSS también pueden crear bloques contenedores, pero no necesitamos desarrollarlos todavía.

`absolute`: sacar una caja del flujo normal

Una caja con `position: absolute` deja de participar en el flujo normal.

```
.etiqueta {
  position: absolute;
  top: 1rem;
  right: 1rem;
}
```

El espacio que habría ocupado como elemento normal ya no se reserva para ella. Las demás cajas se distribuyen como si la caja absoluta no ocupase una posición normal entre ellas. Su ubicación se calcula respecto a su bloque contenedor. Por eso debemos conocer la referencia antes de interpretar `top`, `right`, `bottom` y `left`.



Usos adecuados

- Una pequeña etiqueta superpuesta.
- Un indicador situado sobre una imagen.
- Un elemento decorativo relacionado con un componente.
- Una capa situada en un punto concreto de otro componente.

No utilizar como solución general

Colocar manualmente todos los elementos de una página mediante coordenadas `absolute` dificulta que la interfaz se adapte al contenido y al tamaño disponible. Para construir layouts completos estudiaremos posteriormente Flexbox y Grid.

Aplicación: colocar una etiqueta sobre una tarjeta

Queremos colocar una pequeña etiqueta en la esquina superior derecha de una tarjeta sin modificar la distribución de su contenido.

HTML

```
<div class="tarjeta-curso">
  <span class="estado">Nuevo</span>
  <h2>Diseño de Interfaces Web</h2>
  <p>
    Aprende a diseñar interfaces
    claras, coherentes y accesibles.
  </p>
</div>
```

CSS

```
.tarjeta-curso {
  position: relative;
  padding: 2rem;
  border: 1px solid #E2E8F0;
}

.estado {
  position: absolute;
  top: 1rem;
  right: 1rem;
  padding: 0.35em 0.7em;
}
```

¿Qué ocurre?

1. La tarjeta permanece dentro del flujo normal de la página.
2. `position: relative` establece la referencia que utilizaremos para posicionar el descendiente.
3. `.estado` utiliza `position: absolute` y sale del flujo normal interno.
4. `top: 1rem` y `right: 1rem` se calculan respecto a la referencia establecida por la tarjeta.
5. El contenido principal de la tarjeta no tiene que reservar una fila específica para la etiqueta.

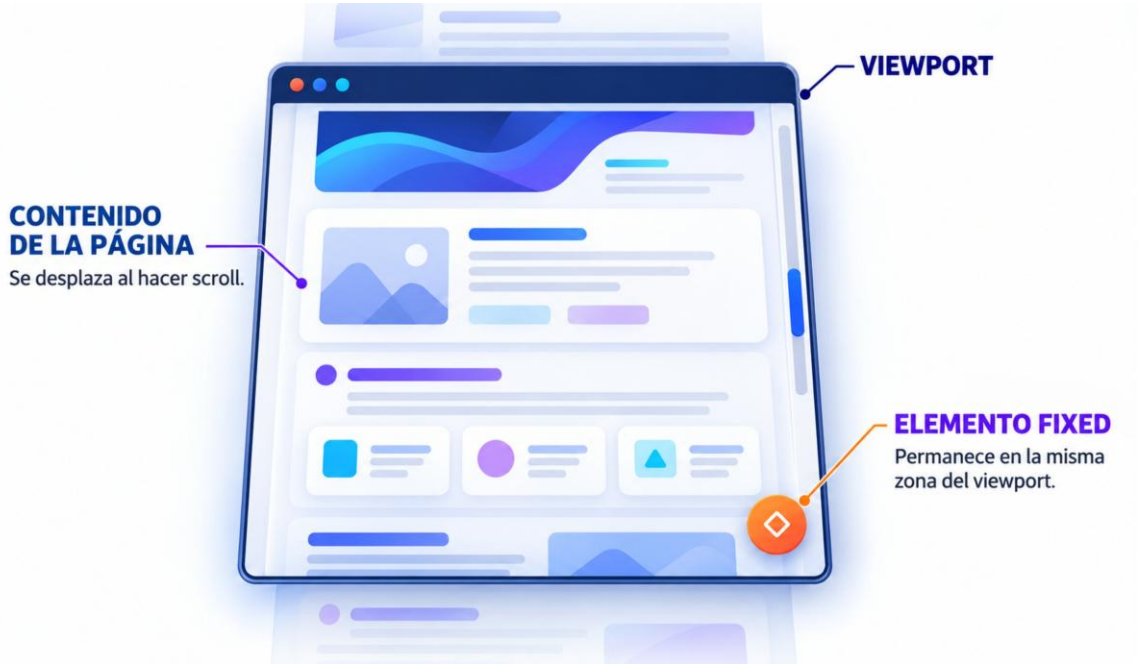
Idea clave: `absolute` funciona bien cuando existe una relación clara entre una capa superpuesta y su componente de referencia.

`fixed`: mantener una caja en una posición durante el scroll

Una caja con `position: fixed` sale del flujo normal. En el caso habitual que estudiaremos aquí, su posición se mantiene respecto al viewport.

```
.ayuda {
  position: fixed;
  right: 1.5rem;
  bottom: 1.5rem;
}
```

Cuando el usuario hace scroll: el documento se desplaza; la caja `.ayuda` permanece a la misma distancia del borde derecho e inferior de la ventana.



Casos habituales

- Un acceso persistente a una herramienta de ayuda.
- Un control flotante.
- Determinados avisos o acciones que deben permanecer disponibles.

Pero `fixed` debe utilizarse con criterio. Un elemento fijo puede tapar contenido, reducir el espacio disponible, interferir con otros controles, o resultar especialmente molesto en pantallas pequeñas.

Idea clave: La permanencia visual de un elemento no justifica por sí sola utilizar `fixed`. Debemos comprobar que no perjudica al contenido ni a la interacción.

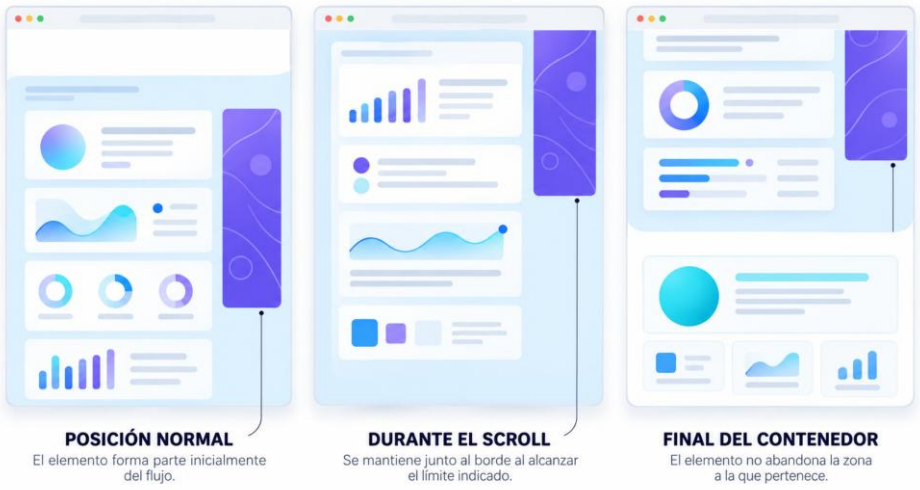
`sticky`: permanecer en el flujo y adherirse durante parte del desplazamiento

`position: sticky` combina características del flujo normal y del posicionamiento desplazado.

```
.indice {
  position: sticky;
  top: 1rem;
}
```

Inicialmente, `.indice` ocupa una posición normal dentro del documento. Cuando el desplazamiento alcanza el límite indicado por `top: 1rem`, el elemento puede mantenerse a esa distancia del borde correspondiente del área de desplazamiento. Sin embargo, no permanece fijado indefinidamente. Su desplazamiento continúa limitado por el contenedor al que pertenece.

Para obtener un comportamiento sticky visible necesitamos establecer al menos un offset apropiado en el eje correspondiente: `top: 0;` o `top: 1rem;`



⚠ **Importante:** El comportamiento `sticky` está relacionado con el contenedor de desplazamiento. Un ancestro que genere un área de scroll mediante `overflow` puede modificar la referencia que utiliza `sticky`. Por eso, cuando `sticky` parece «no funcionar», debemos comprobar también los contenedores y sus propiedades `overflow`.

Caso habitual

Un índice de contenidos puede permanecer visible mientras recorremos una sección y dejar de desplazarse al alcanzar el final de su contenedor.

`fixed` y `sticky` pueden parecer similares, pero resuelven problemas distintos

FIXED

```
.elemento {
  position: fixed;
  top: 1rem;
}
```

- La caja sale del flujo normal.
- En el caso habitual, mantiene su posición respecto al viewport.
- No conserva una posición normal entre los demás elementos.
- Puede permanecer visible durante todo el desplazamiento.

STICKY

```
.elemento {
  position: sticky;
  top: 1rem;
}
```

- La caja participa en el flujo normal.
- Su posición inicial forma parte de la distribución.
- Puede adherirse al alcanzar el límite indicado.
- Continúa condicionada por su zona de desplazamiento y por los límites de su contenedor.

Pregunta 1

¿Debe el elemento formar parte inicialmente del contenido?
Sí → `sticky` puede ser apropiado.

Pregunta 2

¿Debe permanecer asociado directamente a una zona del viewport independientemente de su posición original?
→ `fixed` puede ser apropiado.

Ejemplos

- Botón flotante de ayuda: `fixed`.
- Encabezado de una sección que se mantiene visible mientras recorremos esa sección: `sticky`.

Idea fundamental: No debemos elegir una propiedad porque produzca «algo que se queda quieto». Debemos elegirla según la relación que el elemento mantiene con el flujo y con su referencia.

`z-index`: controlar el orden cuando varias cajas se superponen

Al posicionar elementos podemos hacer que varias cajas ocupen una misma zona. En esas situaciones debemos comprender el orden de apilamiento.

```
.capa-a {
  position: absolute;
  z-index: 1;
}

.capa-b {
  position: absolute;
  z-index: 2;
}
```



Si ambas cajas pueden compararse dentro del mismo contexto de apilamiento, `.capa-b` se pinta por encima de `.capa-a`. Pero debemos evitar una regla falsa: «El elemento con el `z-index` más alto de toda la página siempre aparece delante».

CSS organiza el apilamiento mediante contextos de apilamiento

Determinadas propiedades pueden crear nuevos contextos. Por ejemplo, un elemento posicionado con un `z-index` distinto de `auto` puede establecer uno. Más adelante veremos también otras propiedades que pueden influir en esta estructura. Por ahora necesitamos recordar:

- `z-index` no debe tratarse como una numeración global de toda la página.
- No debemos solucionar los problemas utilizando valores arbitrarios como `z-index: 999999;`
- Debemos comprender primero qué elementos se están superponiendo y a qué contexto pertenecen.

Ejemplo habitual

Una cabecera sticky puede necesitar aparecer por encima del contenido que se desplaza bajo ella.

```
.cabecera {
  position: sticky;
  top: 0;
  z-index: 10;
}
```

`float` y `clear`: comprender una técnica que todavía podemos encontrar

`float` permite desplazar una caja hacia un lado y hacer que el contenido en línea circundante fluya alrededor de ella.

```
.imagen-articulo {
  float: left;
  width: 12rem;
  margin: 0 1rem 1rem 0;
}
```

El texto posterior puede distribuirse alrededor de la imagen flotante.

Valores habituales de `float`

```
float: left;
float: right;
float: none;
```

`clear`

La propiedad `clear` permite impedir que una caja aparezca junto a determinados elementos flotantes.

```
.pie-articulo {
  clear: both;
}
```

Valores habituales: `clear: left;` `clear: right;` `clear: both;`

Uso razonable actual

Hacer que un texto rodee una imagen o un pequeño elemento flotante.

Uso heredado

Durante años se utilizó `float` para construir columnas y layouts completos. No adoptaremos ese método como sistema principal de maquetación. Para distribuir componentes y construir estructuras completas utilizaremos posteriormente Flexbox y Grid.



Idea clave: Conocer `float` es necesario para: entender su funcionamiento real, utilizarlo cuando resulta apropiado, y mantener código existente. No necesitamos convertirlo en nuestra herramienta principal de layout.

Recordatorio: centrar horizontalmente una caja con `margin auto`

En el apartado anterior vimos una técnica fundamental:

```
.contenedor {
  width: 90%;
  max-width: 70rem;
  margin: 0 auto;
}
```

Para comprender por qué funciona debemos analizar el espacio disponible. Supongamos:

1000px

Ancho disponible

700px

Ancho de la caja

300px

Espacio sobrante

1000px − 700px

150px

Cada margen

300px ÷ 2

Si ambos márgenes horizontales utilizan `auto`: `margin-left: auto;` y `margin-right: auto;`, el espacio sobrante se reparte entre ellos. Resultado: margen izquierdo 150px · caja 700px · margen derecho 150px. La caja queda centrada horizontalmente.

También podemos escribir:

```
.contenedor {
  width: min(90%, 70rem);
  margin-inline: auto;
}
```

`margin-inline` representa los márgenes del eje en línea.

⚠ **Importante:** Para que exista un espacio sobrante que repartir, la caja debe tener una anchura menor que el espacio disponible. `margin: 0 auto` no constituye por sí mismo un sistema general para centrar verticalmente una caja.

Centrar un elemento dentro de un contenedor

Supongamos que necesitamos situar una caja posicionada exactamente en el centro de su contenedor.

```
<!-- HTML -->
<div class="pantalla">
  <div class="mensaje">Curso disponible</div>
</div>
```

Empezamos estableciendo la referencia:

```
.pantalla {
  position: relative;
}
```

Después:

```
.mensaje {
  position: absolute;
  top: 50%;
  left: 50%;
}
```



`top: 50%` sitúa la referencia vertical del elemento al 50% de la altura de su bloque contenedor. `left: 50%` sitúa la referencia horizontal al 50% de la anchura de su bloque contenedor. Pero esto NO coloca el centro del elemento en ese punto. Lo que hemos situado en el centro es la esquina de referencia de la caja. Por tanto, la caja comienza en el centro y se extiende hacia la derecha y hacia abajo.

> **El elemento todavía no está centrado. Necesitamos compensar la mitad de sus propias dimensiones.**

`translate(-50%, -50%)`: compensar la mitad de la propia caja

Completamos la técnica anterior para centrar el elemento horizontal y verticalmente respecto a su bloque contenedor:

```
.pantalla {
  position: relative;
}
```

```
.mensaje {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

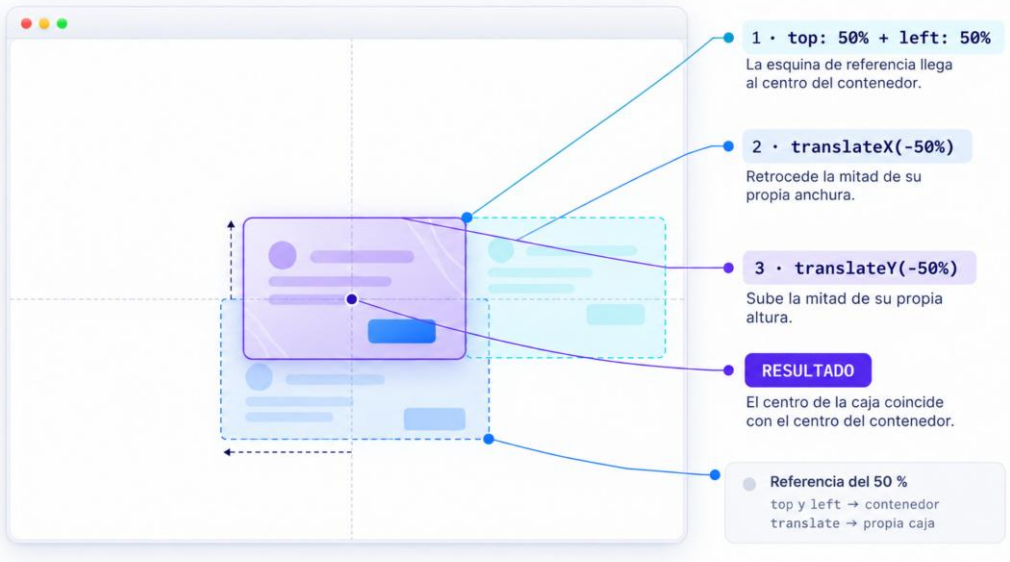
Ahora debemos comprender una diferencia fundamental:

Los porcentajes de `top` y `left`

Se calculan respecto a las dimensiones utilizadas por el bloque contenedor de posicionamiento. Por eso: `left: 50%` nos lleva hasta la mitad horizontal del contenedor. `top: 50%` nos lleva hasta la mitad vertical del contenedor.

Los porcentajes de `translate()`

Se calculan respecto a la propia caja transformada. Por eso: `translateX(-50%)` desplaza la caja hacia la izquierda una distancia equivalente al 50% de su propia anchura. `translateY(-50%)` desplaza la caja hacia arriba una distancia equivalente al 50% de su propia altura.



Idea fundamental: Los dos 50% no utilizan la misma referencia. Ese es precisamente el motivo por el que la técnica funciona. Esta solución resulta útil cuando estamos trabajando con una caja posicionada mediante `absolute`. En 2.6 aprenderemos el centrado mediante Flexbox y Grid.

No existe una única receta para centrar

Antes de elegir una técnica debemos responder: ¿**Qué** queremos centrar? ¿**Respecto a qué** lo queremos centrar? ¿**En qué eje**? ¿Debe el elemento **permanecer en el flujo**?



Caso 1 — Caja de bloque, centrado horizontal

```
.contenedor {
  width: min(90%, 70rem);
  margin-inline: auto;
}
```

Caso 2 — Caja con `position: absolute`, centrado exacto

```
.elemento {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

Caso 3 — Distribución de varios elementos en un layout

No construiremos esa distribución mediante `position: absolute`. En 2.6 utilizaremos Flexbox y Grid porque ofrecen sistemas específicos de distribución y alineación.

`vertical-align` no constituye un sistema general de centrado de cajas. `margin auto` resuelve determinados casos horizontales. `absolute + translate` resulta útil para cajas posicionadas. Flexbox y Grid resolverán posteriormente la mayor parte de los problemas modernos de distribución y alineación.

Caso completo: diferentes problemas requieren diferentes técnicas

Consideremos una página de Aula Abierta con cuatro necesidades distintas: la cabecera debe permanecer visible al desplazarnos por el contenido; una tarjeta debe mostrar una etiqueta superpuesta en su esquina; un panel debe permitir scroll únicamente cuando su contenido sea demasiado largo; un acceso de ayuda debe permanecer en la esquina inferior derecha del viewport.

HTML

```html
<header class="cabecera">
  Aula Abierta
</header>

<div class="tarjeta">
  <span class="estado">Nuevo</span>
  <h2>Diseño de Interfaces Web</h2>
  <p>Curso de 120 horas.</p>
</div>

<div class="panel">
  <p>Sesión 1 · CSS</p>
  <p>Sesión 2 · Posicionamiento</p>
  <p>Sesión 3 · Maquetación</p>
  <p>Sesión 4 · Responsive</p>
  <p>Sesión 5 · Bootstrap</p>
</div>

<a class="ayuda" href="ayuda.html">
  Ayuda
</a>
```

CSS

```css
.cabecera {
  position: sticky;
  top: 0;
  z-index: 10;
}

.tarjeta {
  position: relative;
  padding: 2rem;
}

.estado {
  position: absolute;
  top: 1rem;
  right: 1rem;
}

.panel {
  max-height: 8rem;
  overflow-y: auto;
}

.ayuda {
  position: fixed;
  right: 1rem;
  bottom: 1rem;
}
```

Análisis

- `.cabecera` → `sticky`
  Forma parte inicialmente de la página, pero queremos mantenerla visible durante el desplazamiento.
- `.tarjeta` → `relative`
  Establece la referencia de posicionamiento para el descendiente absoluto.
- `.estado` → `absolute`
  Se superpone en un punto concreto de la tarjeta sin reservar una fila independiente.
- `.panel` → `overflow-y: auto`
  Solo necesitamos desplazamiento vertical cuando el contenido supera su altura máxima.
- `.ayuda` → `fixed`
  Debe mantener una posición estable respecto al viewport.

Idea final: No existe una propiedad «mejor» entre `sticky`, `absolute`, `overflow` o `fixed`. Cada una responde a un problema diferente. La elección correcta depende de la relación que queremos establecer entre el elemento, el flujo y su referencia.

Síntesis: comprender el flujo antes de mover las cajas



> **Idea final.** El posicionamiento no consiste en colocar coordenadas hasta que la interfaz parezca correcta. Una solución sólida comienza identificando: cómo participaría el elemento en el flujo, si debe conservar su espacio, qué referencia necesita, y cómo debe relacionarse con las demás cajas.

En el siguiente apartado estudiaremos cómo dar apariencia visual a estos elementos mediante colores, fondos, tipografía, imágenes e iconos.

Fuentes técnicas

Especificaciones técnicas del W3C / CSS Working Group

CSS Display Module Level 3

Referencia para: `display`; generación de cajas; participación en el flujo; `block`; `inline`; `none`; conceptos *in-flow* y *out-of-flow*.

CSS Positioned Layout Module Level 3

Referencia para: `position`; `static`; `relative`; `absolute`; `fixed`; `sticky`; offsets; bloques contenedores; apilamiento de elementos posicionados.

CSS Overflow Module Level 3

Referencia para: `overflow`; `overflow-x`; `overflow-y`; `visible`; `hidden`; `clip`; `scroll`; `auto`; contenedores de desplazamiento.

CSS Scrollbars Styling Module Level 1

Referencia para: `scrollbar-width`; `scrollbar-color`.

CSS 2 · Visual Formatting Model

Referencia complementaria para: flujo normal; `float`; `clear`; formatos de bloque y en línea; comportamiento clásico del modelo visual de CSS.